

XAMPP & Lokalny Hosting

Wykłady 4

Bartosz Lauks

27 marca 2026



- 1 XAMPP
- 2 Instalacja XAMPP
- 3 Uruchamianie narzędzi XAMPP
- 4 Hostowanie aplikacji
- 5 htaccess

1 XAMPP

2 Instalacja XAMPP

3 Uruchamianie narzędzi XAMPP

4 Hostowanie aplikacji

5 htaccess

XAMPP

Czym jest

- **XAMPP** to darmowe, wieloplatformowe oprogramowanie, które umożliwia szybkie uruchomienie lokalnego serwera WWW. Nazwa XAMPP jest akronimem, który oznacza:
 - **X** – symbolizuje wieloplatformowość (Windows, Linux, macOS).
 - **A** – Apache (serwer WWW).
 - **M** – MySQL/MariaDB (system zarządzania bazami danych).
 - **P** – PHP (język skryptowy).
 - **P** – Perl (kolejny język skryptowy).
- Pozwala na łatwe testowanie i rozwijanie aplikacji internetowych bez potrzeby konfiguracji osobnych serwerów.
- Jest często wykorzystywane przez programistów do lokalnego tworzenia stron internetowych, testowania skryptów oraz pracy z bazami danych.

Dlaczego używamy XAMPP / lokalnego serwera?

Korzyści z lokalnego środowiska

- **Brak kosztów** – nie trzeba od razu kupować hostingu ani VPS.
- **Szybkie testy** – błyskawiczne sprawdzenie zmian w aplikacji bez deploja.
- **Praca offline** – możesz rozwijać aplikację bez połączenia z Internetem.
- **Debugowanie backendu** – pełna kontrola nad PHP, bazą danych i konfiguracją serwera.
- **Bezpieczeństwo** – kod i bazy danych nie są dostępne publicznie.
- **Łatwe eksperymenty** – testowanie konfiguracji serwera, modułów Apache i plików .htaccess bez ryzyka dla środowiska produkcyjnego.

Składniki XAMPP

Apache HTTP Server

- Jest to najpopularniejszy serwer WWW, używany do hostowania stron internetowych.
- Obsługuje protokoły HTTP i HTTPS.
- Pozwala na pełną konfigurację.

MariaDB (wcześniej MySQL)

- Relacyjna baza danych, szeroko stosowana w aplikacjach internetowych.
- Kompatybilna z MySQL, oferuje wysoką wydajność i stabilność.
- Można zarządzać nią przez phpMyAdmin.

Składniki XAMPP

PHP

- Język skryptowy po stronie serwera, używany do tworzenia dynamicznych stron WWW.
- Obsługuje szeroką gamę rozszerzeń i bibliotek.

Perl

- Wieloplatformowy język programowania, używany głównie w skryptach administracyjnych i aplikacjach webowych.

phpMyAdmin

- Graficzny interfejs do zarządzania bazami danych **MySQL/MariaDB**.
- Umożliwia tworzenie, edytowanie i usuwanie baz danych oraz tabel.

Składniki XAMPP

FileZilla FTP Server

- Serwer **FTP**, który umożliwia przesyłanie plików na serwer lokalny.

Mercury (tylko na Windows)

- Serwer poczty e-mail (**SMTP**, **POP3**, **IMAP**), który umożliwia wysyłanie i odbieranie wiadomości e-mail na serwerze lokalnym.
- Jest używany głównie przez programistów do testowania funkcji wysyłania e-maili w aplikacjach **PHP**, bez konieczności korzystania z zewnętrznych usług SMTP, takich jak **Gmail** czy **Outlook**.

Tomcat (tylko na Windows)

- Serwer aplikacji Java, obsługujący aplikacje webowe napisane w **JSP** (**JavaServer Pages**) i **Servletach**.

1 XAMPP

2 Instalacja XAMPP

3 Uruchamianie narzędzi XAMPP

4 Hostowanie aplikacji

5 htaccess

Instalacja XAMPP

Pobieranie i instalacja

- XAMPP można pobrać z oficjalnej strony projektu:

<https://www.apachefriends.org>

Proces instalacji jest prosty i składa się z kilku kroków:

- Pobranie odpowiedniego instalatora dla swojego systemu operacyjnego. (Windows, Linux, MacOS)
- Uruchomienie instalatora i wybór komponentów do instalacji.
- Wskazanie katalogu instalacyjnego.
- Po zakończeniu instalacji można uruchomić XAMPP i skonfigurować serwer.

Instalacja XAMPP

Instalacja w Linux

- W przypadku Windows i MacOS wystarczy tylko uruchomienie instalatora oraz przejście przez wszystkie jego etapy.
- Po instalacji aplikacji możemy ją uruchomić wybierając skrót w **panelu menu**.
- Aby zainstalować XAMPP w Linux przed instalacją musimy wykonać następujące kroki:
 - W pierwszej kolejności należy nadać naszemu plikowi instalacyjnemu prawa do uruchomienia.
`$ sudo chmod +x xampp-linux-*`
 - Teraz możemy uruchomić i przejść przez instalator
`$ sudo ./xampp-linux-*`

Instalacja XAMPP

Uruchamianie w Linux

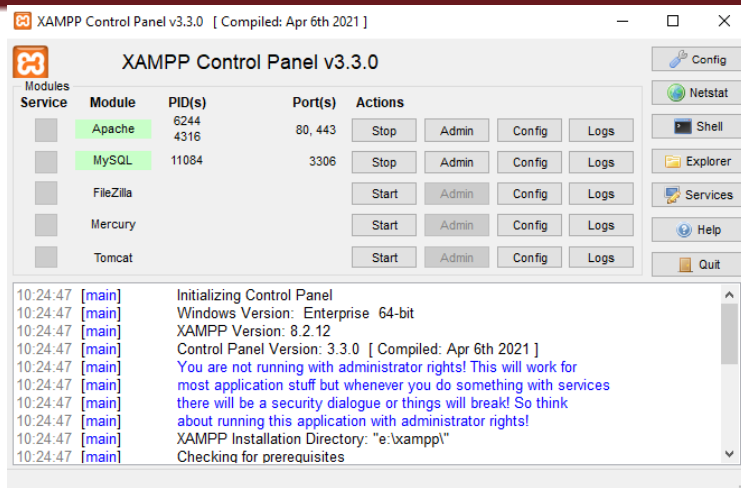
- XAMPP zostanie automatycznie uruchomiony, ale problemem może być z kolejnym uruchomieniem, ponieważ Linux nie tworzy żadnego skrótu do aplikacji.
- Aby uruchomić XAMPP ponownie należy wykonać polecenie w terminalu:
`$ sudo /opt/lampp/manager-linux-x64.run`
- Dla ułatwienia sobie pracy można stworzyć **alias** dla owej komendy oraz zapisać ją w pliku `.bashrc`, aby **BASH** implementował ją przy kolejnym uruchomieniu systemu.

- 1 XAMPP
- 2 Instalacja XAMPP
- 3 Uruchamianie narzędzi XAMPP
- 4 Hostowanie aplikacji
- 5 htaccess

Uruchamianie narzędzi XAMPP

Uruchamianie i testowanie

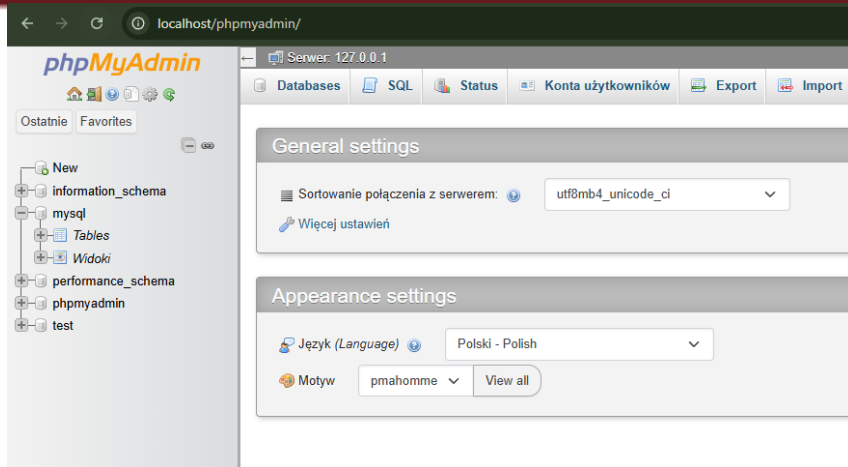
- Po zainstalowaniu oraz uruchomieniu XAMPP, wyświetli się panel kontrolny z serwisami które zawiera narzędzie.
- Do podstawowych potrzeb rozwijania aplikacji web wystarczy uruchomienie serwera **HTTP Apache** oraz **MySQL**, klikając przycisk "**Start**"
- Następnie, aby przetestować działanie Serwera HTTP można wykorzystać przeglądarkę internetową. Sprawdzając adres:
<http://localhost/>
- Jeśli Serwer zwraca nam domyślną stronę internetową XAMPP możemy być pewni że wszystko działa prawidłowo.
- Działanie bazy danych możemy również sprawdzić za pomocą przeglądarki logując się na usługę **phpMyAdmin** pod adresem:
<http://localhost/phpmyadmin>



Rysunek 1: Panel XAMPP w systemie Windows z włączonymi modułami Apache oraz MySQL



Rysunek 2: Domyślna strona wystawiona przez Serwer w XAMPP



Rysunek 3: Panel phpMyAdmin

1 XAMPP

2 Instalacja XAMPP

3 Uruchamianie narzędzi XAMPP

4 Hostowanie aplikacji

5 htaccess

Hostowanie aplikacji

Katalog htdocs

- Katalog **htdocs** to domyślny folder, w którym XAMPP przechowuje pliki stron internetowych.
- Jest on głównym katalogiem serwera Apache, co oznacza, że wszystkie pliki umieszczone w tym folderze mogą być dostępne za pomocą przeglądarki internetowej.
- Serwer Apache, gdy jest uruchomiony, interpretuje pliki znajdujące się w katalogu **htdocs** i udostępnia je w sieci lokalnej lub internecie (jeśli serwer jest skonfigurowany do pracy online).

Hostowanie aplikacji

Mechanizm domyślnego pliku

- Serwer Apache (i inne serwery WWW) mają listę plików, które są domyślnie ładowane, gdy użytkownik odwiedza katalog główny lub podkatalog bez wskazania konkretnego pliku.
- Domyślnie Apache szuka pliku index o jednej z następujących nazw:
 - [index.html](#)
 - [index.php](#)
 - [index.htm](#)
 - [index.asp](#)
 - [index.jsp](#)
- Pierwszy znaleziony plik z tej listy zostaje automatycznie załadowany.

Przykład: localhost/example1

Hostowanie aplikacji

Co jeśli nie ma pliku index?

- Jeśli w katalogu nie znajduje się żaden plik domyślny (np. **index.html**, **index.php**), serwer Apache zachowa następująco:
 - **Wyświetli listę plików w katalogu** (tzw. directory listing)
 - użytkownik zobaczy wszystkie pliki i podkatalogi
 - może wejść w dowolny plik ręcznie
- Pozwala to użytkownikowi na swobodne przeglądanie zawartości katalogu [htdocs](#).
- **W praktyce:** w aplikacjach produkcyjnych listowanie katalogów jest zazwyczaj wyłączone ze względów bezpieczeństwa.

Przykład: localhost/example1NotIndex

Hostowanie aplikacji

Struktura aplikacji a hiperłącza

- Każda aplikacja webowa składa się z wielu stron oraz podstron, między którymi użytkownicy muszą się poruszać. Aby umożliwić nawigację, wykorzystujemy **hiperłącza**.
- Nasze pliki (**.html**, **.php**) znajdują się w katalogu **htdocs**, co oznacza, że struktura katalogów bezpośrednio wpływa na adresy URL aplikacji.
- Wyróżniamy kilka sposobów nawigowania w aplikacji:
 - Nawigacja w tym samym katalogu (np. **about.html**)
 - Nawigacja do podkatalogów (np. **pages/contact.html**)
 - Powrót do katalogu nadrzędnego (np. **../index.html**)
 - Programowa nawigacja w JavaScript

Hostowanie aplikacji

Ścieżki względne i absolutne

- W aplikacjach webowych możemy używać dwóch typów ścieżek:
 - **Ścieżki absolutne** – odnoszą się do pełnej lokalizacji zasobu:
 - </example2/index.html>
 - zależne od domeny (np. localhost)
 - **Ścieżki względne** – odnoszą się do aktualnej lokalizacji pliku:
 - <index.html>
 - <pages/contact.html>
 - <../index.html>
- **Ścieżki względne** są bardziej elastyczne i nie wymagają zmian przy przenoszeniu aplikacji.
- Dzięki temu kod jest bardziej **modułowy i łatwiejszy w utrzymaniu**.

Przykład: localhost/example2

Hostowanie aplikacji

Lokalna domena

- Podczas rozwijania aplikacji webowej często pojawia się potrzeba zasymulowania przyszłej domeny, pod którą będzie znajdować się nasza aplikacja po jej wdrożeniu na serwerze produkcyjnym.
- Taka potrzeba wynika z kilku powodów, może być konieczne uwzględnienie specyficznych ustawień związanych z domeną, takich jak struktura URL, subdomeny, przekierowania, czy kwestie bezpieczeństwa (np. certyfikaty SSL).
- Musimy pamiętać że **localhost** jest domeną ustawioną lokalnie na naszym urządzeniu, a pod jego nazwą kryje się adres **IP 127.0.0.1**.
- Skoro **localhost** jest lokalną domeną oznacza to, że możemy stworzyć kolejne na własne potrzeby.

Hostowanie aplikacji

Jak dodać lokalną domenę do systemu ?

- W systemach operacyjnych jak Windows, macOS i Linux, proces dodawania lokalnej domeny jest podobny, ale zależy od systemu plików i narzędzi używanych do konfiguracji.

Windows

- Przejdź do edycji pliku
C : \Windows\System32\drivers\etc\hosts
- W pliku hosts dodaj wiersz, który przypisuje nazwę domeny do adresu IP lokalnego hosta.
127.0.0.1 www.moja-domena.pl

Hostowanie aplikacji

MacOS & Linux

- Przejdź do edycji pliku jako superuser
`$ sudo nano /etc/hosts`
- Podobnie jak w przypadku systemu Windows, dodaj wiersz przypisujący lokalną domenę do adresu **IP 127.0.0.1**.
`127.0.0.1 www.moja-domena.pl`

Testowanie lokalnej domeny

- Teraz użyj dodanej domeny w przeglądarce, aby sprawdzić czy konfiguracja się powiodła, powinieneś zobaczyć stronę główną swojej aplikacji.
- Jeśli nie zadeklarowałeś ścieżek absolutnych w hiperłączach, zmiana nazwy domeny nie wpłynie na możliwość nawigacji między podstronami.
- Nazwa domeny zostanie dynamicznie zmieniona przed ścieżką do zasobu.

- 1 XAMPP
- 2 Instalacja XAMPP
- 3 Uruchamianie narzędzi XAMPP
- 4 Hostowanie aplikacji
- 5 htaccess

htaccess

Co to jest i do czego służy?

- Plik **.htaccess** (**H**ypertext **A**ccess) to specjalny plik konfiguracyjny używany przez serwer **Apache** do definiowania reguł i ustawień dla katalogów oraz plików na serwerze.
- Jest on niezwykle przydatny do zarządzania dostępem do zasobów, przekierowaniami, bezpieczeństwem oraz optymalizacją działania stron internetowych.

htaccess

Lokalizacja i sposób działania

- Plik **.htaccess** umieszczany jest w katalogu głównym strony internetowej lub w podkatalogach, jeśli chcemy stosować różne reguły dla różnych części witryny.
- Serwer Apache odczytuje plik **.htaccess** i stosuje w nim zawarte instrukcje przy każdym żądaniu HTTP.
- W kolejnych slajdach zostaną opisane Najczęściej używane dyrektywy w pliku **.htaccess**.

htaccess

Blokowanie directory listing

- Konfiguracja **.htaccess** pozwala na zablokowanie directory listing, który umożliwia przeglądanie zawartości katalogu [htdocs](#).
- Wystarczy dodać dyrektywę **Options -Indexes**, aby serwer HTTP zwrócił błąd **403 Forbidden**.
- Jeśli listowanie katalogów jest wyłączone, użytkownik nie ma dostępu do zawartości katalogu.
- Istnieje jeszcze dyrektywa **DirectoryIndex**, która ustawia kolejność zwracania domyślny stron w katalogu.

Listing 1: Blokowanie directory listing

```
1  # Blokowanie listowania katalogów
2  Options -Indexes
3
4  # Ustawienie strony startowej katalogu
5  # Serwer najpierw szuka index.html, jeśli go nie ma to
   indexDirectoryIndex.html
6  DirectoryIndex index.html indexDirectoryIndex.html
```

Przekierowania URL

- Przekierowanie URL (ang. URL Redirection lub URL Forwarding) to mechanizm, który automatycznie przenosi użytkownika z jednej strony na inną. Jest to używane w wielu przypadkach, np.:
 - **Zmiana adresu strony** – gdy strona przenosi się na nowy URL. (Przekierowanie trwałe 301)
 - **Korekta błędów użytkownika**– np. przekierowanie z example.com do www.example.com.
 - **Zarządzanie ruchem** – np. tymczasowe przekierowanie na stronę konserwacji. (Przekierowanie tymczasowe 302)
 - **SEO** – np. przekierowanie HTTP na HTTPS, aby poprawić bezpieczeństwo i pozycjonowanie w Google.

Listing 2: Przykłady przekierowania URL

```
1  #Przekierowanie 301 (trwale) - zmiana domeny
2  Redirect 301 /stara-strona.html https://example.com/nowa-strona.html
3
4  #Lub w bardziej zaawansowanej formie:
5  RewriteEngine On
6  RewriteRule ^stara-strona\.html$ https://example.com/nowa-strona.html [
    R=301,L]
7
8  #Przekierowanie całej domeny na nowa
9  RewriteEngine On
10 RewriteCond %{HTTP_HOST} ^stara-domena\.com$ [OR]
11 RewriteCond %{HTTP_HOST} ^www\.stara-domena\.com$
12 RewriteRule (.*?)$ https://www.nowa-domena.com/$1 [R=301,L]
13
14 #Przekierowanie HTTP -> HTTPS
15 RewriteEngine On
16 RewriteCond %{HTTPS} off
```

```
17 RewriteRule ^(.*)$ https://%{HTTP_HOST}/$1 [R=301,L]
18
19 #Przekierowanie domeny z www. na bez www.
20 RewriteEngine On
21 RewriteCond %{HTTP_HOST} ^www\.(.*)$ [NC]
22 RewriteRule ^(.*)$ https://%1/$1 [R=301,L]
```

Przepisywanie adresów URL (URL Rewriting)

- Przepisywanie adresów URL (ang. URL Rewriting) to technika, która pozwala na dynamiczne modyfikowanie adresów URL w przeglądarce użytkownika bez zmiany struktury plików na serwerze.
- Jest często stosowana do:
 - **Tworzenia przyjaznych dla użytkownika i SEO URL-i** (np. `example.com/produkt/123` zamiast `example.com?produkt=123`).
 - **Ukrywania struktury plików** (np. `example.com/kontakt` zamiast `example.com/contact.php`).
 - **Obsługi aliasów** (np. `old-page.html` → `nowa-strona`).

Listing 3: Przykłady Przepisywanie adresów URL

```
1  #Przepisywanie prostej sciezki URL
2  #example.com/projekt.php?id=5 -> example.com/projekt/5
3  RewriteEngine On
4  RewriteRule ^projekt/([0-9]+)$ projekt.php?id=$1 [L]
5
6  #Usuwanie rozszerzen plikow (np. .php, .html)
7  #example.com/kontakt -> example.com/kontakt.php
8  RewriteEngine On
9  RewriteCond %{REQUEST_FILENAME} !-d
10 RewriteCond %{REQUEST_FILENAME}\.php -f
11 RewriteRule ^(.*)$ $1.php [L]
12
13 # Aliasy
14 RewriteEngine On
15 RewriteRule o-mnie omnie.html
16 RewriteRule oferta-pracy-dla-programisty-backend index.html
```

Ochrona plików i katalogów

- W pliku [.htaccess](#) możesz łatwo kontrolować, kto ma dostęp do określonych plików lub katalogów oraz jak chronić te zasoby przed nieautoryzowanym dostępem.
- Rodzaje ochrony plików i katalogów to:
 - Ochrona przed dostępem z zewnątrz
 - Ochrona za pomocą hasła
 - Zabezpieczenie przed wlewnem złośliwego oprogramowania

Listing 4: Przykłady Ochrona plików i katalogów

```
1  # Jeśli chcesz zablokować dostęp do konkretnego katalogu, gdzie  
   znajduje się .htaccess  
2  <IfModule mod_authz_core.c>  
3      Require all denied  
4      Require ip 192.168.1.100  
5  </IfModule>  
6  
7  # Ochrona plików przed dostępem  
8  <Files "config.php">  
9      Require all denied  
10 </Files>  
11  
12 # Ochrona katalogu private hasłem  
13 AuthType Basic  
14 AuthName "Restricted Access"  
15 AuthUserFile /sciezka/do/.htpasswd  
16 Require valid-user
```

```
17
18 # Blokowanie dostępu do plików o określonych rozszerzeniach
19 <FilesMatch "\.(php|cgi|pl)$"> # dopasowuje pliki z rozszerzeniami .php
    , .cgi, .pl
20     Require all denied
21 </FilesMatch>
22
23 # Blokowanie witryny przed botami
24 RewriteEngine On
25 RewriteCond %{HTTP_USER_AGENT} ^nazwabota [OR]
26 RewriteCond %{HTTP_USER_AGENT} ^nazwabota
27 RewriteRule .* - [F]
28
29 # Ochrona przed hotlinkingiem (blokowanie użycia plików przez inne
    strony)
30 RewriteEngine On
31 RewriteCond %{HTTP_REFERER} !^$
32 RewriteCond %{HTTP_REFERER} !^https?://(www\.)?twojadomena\.com [NC] #
    sprawdza, czy odwołanie pochodzi z Twojej domeny.
```

```
33 RewriteRule \.(jpg|jpeg|png|gif)$ - [F] # blokuje dostep do obrazków .  
    jpg, .jpeg, .png, .gif z innych domen.
```


Strony błędów

- W pliku `.htaccess` możesz skonfigurować własne strony błędów dla różnych typów błędów HTTP.
- Aby uniknąć wyświetlania domyślnych storny błędów przez serwer HTTP, wystarczy dodać odpowiednie reguły w pliku `.htaccess`.

Listing 5: Przykłady Strony błędów

```
1  # Strona błędu 404 - Not Found
2  ErrorDocument 404 /404.html
3
4  # Strona błędu 403 - Forbidden
5  ErrorDocument 403 /403.html
6
7  # Strona błędu 500 - Internal Server Error
8  ErrorDocument 500 /500.html
9
10 # Strona błędu 502 - Bad Gateway
11 ErrorDocument 502 /502.html
12
13 # Strona błędu 503 - Service Unavailable
14 ErrorDocument 503 /503.html
```

Blokowanie dostępu z określonych adresów IP

- Blokowanie dostępu do strony lub określonych zasobów na podstawie adresu IP jest jednym ze sposobów ochrony serwera przed niechcianymi użytkownikami, botami, próbami ataków (np. brute force) lub innymi zagrożeniami.
- Gdy użytkownik próbuje uzyskać dostęp do Twojej strony, jego adres IP jest sprawdzany. Jeśli ten adres znajduje się na liście zablokowanych, dostęp do strony lub zasobu jest automatycznie odrzucany.

Listing 6: Przykład Blokowanie dostępu z określonych adresów IP

```
1  # Blokowanie dostępu z pojedynczych adresów IP
2  Require not ip 192.168.1.1
3  Require not ip 203.0.113.42
4  Require not ip 198.51.100.100
5
6  # Blokowanie całej sieci 192.168.0.0/24
7  Require not ip 192.168.0.0/24
8
9  # Zezwolenie na dostęp wszystkim innym
10 Require all granted
11
12 # Te same procedury, ale w tagach:
13 <RequireAll>
14     Require all granted
15     Require not ip 192.168.1.1
16     Require not ip 203.0.113.42
17     Require not ip 198.51.100.100
```

```
18     Require not ip 192.168.0.0/24
19 </RequireAll>
```

Cache przeglądarki i mod_expires

- **Cache przeglądarki** to mechanizm przechowywania zasobów stron internetowych (np. obrazków, plików CSS, JavaScript) na komputerze użytkownika.
- Dzięki temu przeglądarka nie musi pobierać tych zasobów za każdym razem, gdy użytkownik odwiedza stronę, co przyspiesza ładowanie strony i zmniejsza obciążenie serwera.
- Moduł **mod_expires** umożliwia ustawienie nagłówków HTTP informujących przeglądarkę o czasie przechowywania plików w pamięci podręcznej.
- Pozwala to na kontrolowanie, jak długo przeglądarka ma przechowywać pliki przed ich ponownym załadowaniem.

Listing 7: Przykłady użycia mod_expires

```
1  # Włączenie mod_expires
2  ExpiresActive On
3
4  # Przechowywanie plików statycznych przez długi czas (1 rok)
5  ExpiresByType image/jpeg "access plus 1 year"
6  ExpiresByType image/png "access plus 1 year"
7  ExpiresByType image/gif "access plus 1 year"
8  ExpiresByType text/css "access plus 1 year"
9  ExpiresByType application/javascript "access plus 1 year"
10 ExpiresByType application/font-woff2 "access plus 1 year"
11 ExpiresByType application/font-woff "access plus 1 year"
12 ExpiresByType application/font-ttf "access plus 1 year"
13
14 # Przechowywanie plików HTML przez krótki czas (5 minut)
15 ExpiresByType text/html "access plus 5 minutes"
16
17 # Domyślnie przechowywanie przez 1 dzień dla plików, które nie mają
```



określonego typu MIME

18 ExpiresDefault "access plus 1 day"

19

20 *# Wyłączenie cache dla plików PHP*

21 <FilesMatch "\.php\$">

22 Header set Cache-Control "no-store, no-cache, must-revalidate"

23 </FilesMatch>

24

25 *# Dodatkowe ustawienia nagłówków Cache-Control*

26 *# max-age przetrzymywanie plików w cache przez 1 godzinę*

27 Header set Cache-Control "max-age=3600"

Dziękuję za uwagę !

Bartosz Lauks